

Protect any repository

SecureGate guards its own repository with two gates, and any GitHub repository can have the same two:

- **The merge gate:** a check on every pull request, which turns red and locks the merge button when a pull request brings a secret. It is the GitHub Action in this repository, `action.yml`.
- **The laptop gate:** a check before every commit, which stops the commit on your own computer.

Use both. The laptop gate stops most mistakes before anyone sees them; the merge gate also catches what reaches GitHub anyway, for example after `git commit --no-verify`.

The merge gate: one workflow file

Step 1. Add the workflow. In your repository, add the file `.github/workflows/secret-gate.yml`:

```
name: secret-gate

on:
  pull_request:
    branches: [main]

permissions:
  contents: read          # read the code
  pull-requests: write    # post SecureGate's comment
  security-events: write  # send the findings to the Security tab

jobs:
  secret-gate:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          fetch-depth: 0          # every commit of the pull request
          persist-credentials: false
      - uses: Bhavik-Kadian/Nexora@v1.0.0
```

Step 2. Open a pull request. The check **secret-gate** runs for a few minutes. It scans every commit of the pull request with Gitleaks, TruffleHog, Semgrep and Bandit, posts one comment (and updates it after each push), shows a summary on the check's page, and sends the findings to the **Security** tab. A blocked secret turns the check red.

Step 3. Make the check required, so that a red check locks the merge button: follow The one-time setting (merge-gate.pdf) in your repository. The check only appears in GitHub's list after it has run once, so open the pull request of step 2 first.

What the action needs: a Linux runner, the checkout with `fetch-depth: 0`, and the three permissions above. Without the full history it stops at once and says so, and the check fails. It installs Python 3.12 into the job, so give it a job of its own, as above.

Why @v1.0.0: the tag decides which SecureGate runs, with its scanners at the versions it was tested with. Everything SecureGate downloads is pinned: Gitleaks and TruffleHog are checked against their published checksums, and Semgrep, Bandit and SecureGate's own packages are installed at exact versions. For a pin that nobody can move, use the commit of the release instead of the tag.

Choose what it does

Input	Default	What it does
<code>policy</code>	empty	Your policy file. Empty: your <code>policy.yaml</code> when the base branch has one, otherwise SecureGate's own. A file you name here must already be on the base branch, or the check fails.
<code>comment</code>	<code>true</code>	Post one comment on the pull request and update it after every push.
<code>sarif</code>	<code>true</code>	Send the findings to the Security tab.
<code>github-token</code>	the job's own token	The token that posts the comment.
<code>ai-endpoint</code> , <code>ai-deployment</code> , <code>ai-key</code>	empty	Ask SecureGate's AI agents for advice, added to the comment and the summary. Without <code>ai-key</code> they are not asked. See The AI agents (agents.pdf).

The action gives back `exit-code` (0 = pass, 1 = a secret is blocked, 2 = the scan could not finish) and `findings`, the path of `findings.json`, which holds masked values only.

With the AI agents, the last step becomes:

```
- uses: Bhavik-Kadian/Nexora@v1.0.0
  with:
    ai-endpoint: ${vars.SECUREGATE_AI_ENDPOINT}
    ai-deployment: ${vars.SECUREGATE_AI_DEPLOYMENT}
    ai-key: ${secrets.SECUREGATE_AI_KEY}
```

Add the key as a repository **secret** and the other two as **variables** (Settings, then Secrets and variables, then Actions). The agents advise; the policy still decides, and they never see a whole secret.

Your own rules

Without a `policy.yaml` of your own, SecureGate's [policy.yaml](#) decides: it blocks payment, cloud and private keys and keys that a provider confirms are live, warns about passwords in code and about values in tests and docs, and ignores placeholders such as `changeme`. To change it, copy it to the root of your repository, edit it (the comments at its top explain every field) and merge it.

The action always reads the policy **as it is on the pull request's base branch**. A pull request that edits the policy is judged by the old rules, and the new ones count from the next pull request. So a pull request cannot loosen the rules that judge it.

Which key shapes the scanners look for (`.gitleaks.toml`, `.trufflehog.yaml` and `rules/securegate-risky.yaml`) comes from the action itself.

What it cannot do

- **Pull requests from forks** get no comment and no Security-tab results, because GitHub gives them a read-only token. The check still runs and decides.

- **A pull request can edit the workflow that runs the check**, in any repository. Ask for a review of every change to `.github/` (see "The merge gate" in Limitations (limitations.pdf)).
- **It judges pull requests only.** Run on any other event, it stops with exit code 2 and says why.

The laptop gate: before every commit

You need Python 3.12, Git and Gitleaks 8.30.1 (on Windows: `winget install --id Gitleaks.Gitleaks -e --version 8.30.1`).

Step 1. Install SecureGate and pre-commit, at the versions they were tested with:

```
pip install "securegate @ git+https://github.com/Bhavik-Kadian/Nexora@v1.0.0" pre-commit==4.6.2 -c https://raw.githubusercontent.com/Bhavik-Kadian/Nexora/v1.0.0/constraints.txt
```

Install them where your shell finds them when you commit, or write the full path of `securegate` in step 3.

Step 2. Copy the rules. Copy `policy.yaml` and `.gitleaks.toml` from [SecureGate v1.0.0](#) to the root of your repository, and add `.securegate/` to your `.gitignore`: the laptop gate keeps its fingerprint key and its last report there.

Step 3. Add the hook to `.pre-commit-config.yaml`, at the root of your repository:

```
repos:
  - repo: local
    hooks:
      - id: securegate
        name: SecureGate secret scan (staged changes)
        language: system
        entry: securegate scan . --mode staged --out .securegate/staged-findings.json
        pass_filenames: false
        always_run: true
        stages: [pre-commit]
```

Step 4. Turn it on with `pre-commit install`, once, in your repository.

From then on, every `git commit` scans what you staged. A blocked secret stops the commit and shows the file, the line, the masked value, why it was blocked and how to fix it. Without `policy.yaml` or `.gitleaks.toml` the scan fails with exit code 2, and the commit stops too, because SecureGate never passes without looking.

Check it worked

- **Merge gate:** open a pull request that adds a fake key. `securegate demo-token` prints a made-up ACME Pay token that is safe for this: the check turns red with rule 8 and the comment shows the token masked. Close that pull request without merging it.
- **Laptop gate:** stage a file with the same fake token and run `git commit`: SecureGate stops the commit. Then unstage it with `git restore --staged <file>` and delete the file.

Never test with a real key: a real key in a pull request is leaked, even when the check is red.